

1a)

I randomly selected 3 rows from my dataframe and printed out the rows to get the review_body and star_rating values.

Then, I obtained rating statistics using pandas' value_counts() function on the star_rating column.

Sample Review	Rating
Great product.	5
Gold plated fusers are the best! It will never fail on you	5
Gorgeous colors and easy to use	4

Rating	Count
5	1580941
4	417975
3	193452
2	138215
1	306576

1b)

I first created a new "sentiment" column with default value of -1. Then, I used df.loc to set sentiment of all reviews with star_rating ≥ 4 to 1 and star_rating ≤ 2 to 0.

Sentiment	Count
Positive (1)	1998916
Negative (0)	444791
Neutral (-1)	193452

2)

I performed data cleaning using a combination of pandas builtin functions, BeautifulSoup and regex. I defined as many contractions as I could think of, and additionally added mappings for contractions without apostrophes as I expect many people to write contractions the lazy way in their reviews.

	Average Character Length
Before cleaning	319
After cleaning	305

3)

I removed stopwords using nltk's set of English stopwords and excluded words like will, would, etc. as well as no and not, as they are important to preserving sentiment. I then performed lemmatization using nltk's WordNetLemmatizer

Review comparison before/after data cleaning + preprocessing	
Before	After
I wish I had read the reviews before I purchased them. It didn't ruin the printer but they didn't work and I needed to get some photos printed. I am extremely disappointed.	wish have read review purchase do not ruin printer do not work need get photos print be extremely disappoint
The day of the week is automatically populated when you enter the date. But it is set for 2008, and there is no way to change it to the correct day of the week. I set it on Saturday, and there was no way to get it to say Saturday. It will only have the correct day of the week once every seven years. I don't see any advantage to the magnetic key holders over simple hooks. Other than you are forced to put your keys on the magnetic key ring that comes with the product. Because we all need one more thing in our pockets. I wouldn't buy this again.	day week be automatically populate enter date be set be no way change correct day week set saturday no way get say saturday will have correct day week every seven years do not see advantage magnetic key holders simple hook be force put key magnetic key ring come product need one thing pocket would not buy

Lasted 2 weeks and no longer works	last weeks no longer work
------------------------------------	---------------------------

	Average Character Length
Before preprocessing	305
After preprocessing	201

5) Perceptron

Perceptron Training Accuracy: 0.9948

Perceptron Training Precision: 0.9908

Perceptron Training Recall: 0.9990

Perceptron Training F1-score: 0.9949

Perceptron Testing Accuracy: 0.8930

Perceptron Testing Precision: 0.8934

Perceptron Testing Recall: 0.8923

Perceptron Testing F1-score: 0.8929

All 4 test metrics have approximately the same value for the perceptron model.

6) SVM

SVM Training Accuracy: 0.9952

SVM Training Precision: 0.9908

SVM Training Recall: 0.9997

SVM Training F1-score: 0.9952

SVM Testing Accuracy: 0.8837

SVM Testing Precision: 0.8639

SVM Testing Recall: 0.9109

SVM Testing F1-score: 0.8867

SVM resulted in higher recall than precision, meaning it produced fewer false negatives but more false positives.

7) Logistic Regression

Logistic Regression Training Accuracy: 0.9887
Logistic Regression Training Precision: 0.9804
Logistic Regression Training Recall: 0.9975
Logistic Regression Training F1-score: 0.9888
Logistic Regression Testing Accuracy: 0.8985
Logistic Regression Testing Precision: 0.8767
Logistic Regression Testing Recall: 0.9276
Logistic Regression Testing F1-score: 0.9014

Logistic regression resulted in higher recall than precision, meaning it produced fewer false negatives but more false positives.

8) Multinomial Naive Bayes

Naive Bayes Training Accuracy: 0.9457
Naive Bayes Training Precision: 0.9794
Naive Bayes Training Recall: 0.9106
Naive Bayes Training F1-score: 0.9438
Naive Bayes Testing Accuracy: 0.8688
Naive Bayes Testing Precision: 0.8870
Naive Bayes Testing Recall: 0.8452
Naive Bayes Testing F1-score: 0.8656

Naive Bayes resulted in lower recall than precision, meaning it produced more false negatives but fewer false positives.



```
In [209... import pandas as pd
import numpy as np
import nltk
nltk.download('wordnet')
import re
from bs4 import BeautifulSoup
from sklearn.model_selection import train_test_split
```

```
[nltk_data] Downloading package wordnet to /Users/paul/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

```
In [210... ! pip install bs4 # in case you don't have it installed
```

```
# Dataset: https://s3.amazonaws.com/amazon-reviews-pds/tsv/amazon_reviews_us_E
#           https://web.archive.org/web/20201127142707if_/https://s3.amazonaws.
```

```
Requirement already satisfied: bs4 in /Users/paul/Desktop/cs544/venv/lib/python
3.14/site-packages (0.0.2)
```

```
Requirement already satisfied: beautifulsoup4 in /Users/paul/Desktop/cs544/ven
v/lib/python3.14/site-packages (from bs4) (4.14.3)
```

```
Requirement already satisfied: soupsieve>=1.6.1 in /Users/paul/Desktop/cs544/ve
nv/lib/python3.14/site-packages (from beautifulsoup4->bs4) (2.8.3)
```

```
Requirement already satisfied: typing-extensions>=4.0.0 in /Users/paul/Desktop/
cs544/venv/lib/python3.14/site-packages (from beautifulsoup4->bs4) (4.15.0)
```

```
[notice] A new release of pip is available: 25.3 -> 26.0
```

```
[notice] To update, run: pip install --upgrade pip
```

Dataset Preparation

Read Data

```
In [211... filepath = "data.tsv"

df = pd.read_csv(filepath, sep='\t', engine="python", on_bad_lines="skip")

print(df.head())
print(df.shape)
```

	marketplace	customer_id	review_id	product_id	product_parent	\
0	US	43081963	R18RVCKGH1SSI9	B001BM2MAC	307809868	
1	US	10951564	R3L4L6LW1PU0FY	B00DZYEXPQ	75004341	
2	US	21143145	R2J8AWXWTDX2TF	B00RTMUHDW	529689027	
3	US	52782374	R1PR37BR7G3M6A	B00D7H8XB6	868449945	
4	US	24045652	R3BDDDDZMZBZDPU	B001XCWP34	33521401	

	product_title	product_category	\
0	Scotch Cushion Wrap 7961, 12 Inches x 100 Feet	Office Products	
1	Dust-Off Compressed Gas Duster, Pack of 4	Office Products	
2	Amram Tagger Standard Tag Attaching Tagging Gu...	Office Products	
3	AmazonBasics 12-Sheet High-Security Micro-Cut ...	Office Products	
4	Derwent Colored Pencils, Inktense Ink Pencils,...	Office Products	

	star_rating	helpful_votes	total_votes	vine	verified_purchase	\
0	5	0	0	N	Y	
1	5	0	1	N	Y	
2	5	0	0	N	Y	
3	1	2	3	N	Y	
4	4	0	0	N	Y	

	review_headline	\
0	Five Stars	
1	Phfffffft, Phfffffft. Lots of air, and it's C...	
2	but I am sure I will like it.	
3	and the shredder was dirty and the bin was par...	
4	Four Stars	

	review_body	review_date
0	Great product.	2015-08-31
1	What's to say about this commodity item except...	2015-08-31
2	Haven't used yet, but I am sure I will like it.	2015-08-31
3	Although this was labeled as "new" the...	2015-08-31
4	Gorgeous colors and easy to use	2015-08-31

(2637159, 15)

Keep Reviews and Ratings

```
In [212... cols = ["review_body", "star_rating"]
df = df[cols]
```

```
In [213... with pd.option_context('display.max_colwidth', None):
print(df.iloc[0])
```

```
review_body    Great product.
star_rating      5
Name: 0, dtype: object
```

```
In [214... with pd.option_context('display.max_colwidth', None):
print(df.iloc[4])
```

```
review_body    Gorgeous colors and easy to use
star_rating    4
Name: 4, dtype: object
```

```
In [215... with pd.option_context('display.max_colwidth', None):
           print(df.iloc[6])
```

```
review_body    Gold plated fusers are the best! It will never fail on you
star_rating    5
Name: 6, dtype: object
```

```
In [216... # 1a) rating statistics
rating_counts = df['star_rating'].value_counts()
print(rating_counts)
```

```
star_rating
5    1580941
4     417975
1     306576
3     193452
2     138215
Name: count, dtype: int64
```

Relabeling and Sampling

First form three classes and print their statistics. Then randomly select 100,000 reviews from the positive and 100,000 reviews from the negative

```
In [217... # create binary labels
df["sentiment"] = -1
df.loc[df["star_rating"] >= 4, "sentiment"] = 1
df.loc[df["star_rating"] <= 2, "sentiment"] = 0

# count sentiment classes
sentiment_counts = df["sentiment"].value_counts()
print(sentiment_counts)
```

```
sentiment
1    1998916
0     444791
-1     193452
Name: count, dtype: int64
```

```
In [218... # discard rating 3 reviews (-1 sentiment)
df = df[df["sentiment"] != -1]
sentiment_counts = df["sentiment"].value_counts()
print(sentiment_counts)
print(df.columns)
```

```
sentiment
1    1998916
0     444791
Name: count, dtype: int64
Index(['review_body', 'star_rating', 'sentiment'], dtype='str')
```

```
In [219... # randomly select 100,000 positive and 100,000 negative sentiment reviews
n = 100000

df = (
    df.groupby("sentiment", group_keys=False)
      .sample(n=n, random_state=42)
)

print(df["sentiment"].value_counts())
```

sentiment
0 100000
1 100000
Name: count, dtype: int64

Data Cleaning

```
In [220... # print average length of reviews by char length BEFORE cleaning
avg_len = df["review_body"].str.len().mean()
print("Avg len before cleaning:", round(avg_len))

# print 3 sample reviews BEFORE cleaning and preprocessing
with pd.option_context('display.max_colwidth', None):
    print(df["review_body"][:3])

# convert all reviews to lowercase
df["review_body"] = df["review_body"].str.lower()

def strip_html_and_urls(text):
    if pd.isna(text):
        return text
    # Remove HTML
    text = BeautifulSoup(text, "html.parser").get_text()
    # Remove URLs
    text = re.sub(r'http\S+|www.\S+', '', text)
    return text

# remove extra spaces
df["review_body"] = df["review_body"].str.replace(r"\s+", " ", regex=True)

# perform contractions
contractions = {
    "won't": "will not",
    "wouldn't": "would not",
    "can't": "cannot",
    "couldn't": "could not",
    "i'm": "i am",
    "ain't": "am not",
    "it's": "it is",
    "you're": "you are",
    "they're": "they are",
    "i've": "i have",
}
```



```

    "isn't": "is not",
    "aren't": "are not",
    "don't": "do not",
    "didn't": "did not",
    "doesn't": "does not",
    "musn't": "must not",
    "hasn't": "has not",
    "hadn't": "had not",
    "haven't": "have not"
}

# add the lazy version of contractions (without apostrophe)
# to dict
for k in list(contractions.keys()):
    contractions[k.replace("'", "")] = contractions[k]

def expand_contractions(text):
    if pd.isna(text):
        return text
    pattern = re.compile(r'\b(' + '|'.join(contractions.keys()) + r')\b')
    return pattern.sub(lambda x: contractions[x.group()], text)

df["review_body"] = df["review_body"].apply(expand_contractions)

# remove non-alphabetical characters
df["review_body"] = df["review_body"].str.replace(r"[^a-zA-Z\s]", "", regex=True)

# print average length of reviews by char length AFTER cleaning
avg_len = df["review_body"].str.len().mean()
print("Avg len after cleaning:", round(avg_len))

```

Avg len before cleaning: 319

1862138

I wish I had read the reviews before I purchased them. It didn't ruin the printer but they didn't work and I needed to get some photos printed. I am extremely disappointed.

2468360 The day of the week is automatically populated when you enter the date. But it is set for 2008, and there is no way to change it to the correct day of the week. I set it on Saturday, and there was no way to get it to say Saturday. It will only have the correct day of the week once every seven years.

I don't see any advantage to the magnetic key holders over simple hooks. Other than you are forced to put your keys on the magnetic key ring that comes with the product. Because we all need one more thing in our pockets. I wouldn't buy this again.

1229465

Lasted 2 weeks and no longer works

Name: review_body, dtype: str

Avg len after cleaning: 305

Pre-processing

remove the stop words

```
In [221... from nltk.corpus import stopwords
nltk.download("stopwords")

# print average length of reviews by char length BEFORE preprocessing
avg_len = df["review_body"].str.len().mean()
print("Avg len before preprocessing:", round(avg_len))

stop_words = set(stopwords.words("english"))
words_to_keep = {
    "not", "no",
    "will", "would", "can", "could", "am", "is", "are",
    "have", "has", "had", "must", "do", "does", "did"
}
stop_words -= words_to_keep
df["review_body"] = df["review_body"].apply(
    lambda x: " ".join([word for word in x.split() if word not in stop_words])
    if isinstance(x, str) else ""
)
```

Avg len before preprocessing: 305

```
[nltk_data] Downloading package stopwords to /Users/paul/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

perform lemmatization

```
In [222... from nltk.stem import WordNetLemmatizer
nltk.download("wordnet")
nltk.download("omw-1.4")

lemmatizer = WordNetLemmatizer()
df["review_body"] = df["review_body"].apply(lambda x: " ".join([lemmatizer.lem

# print 3 sample reviews AFTER cleaning and preprocessing
with pd.option_context('display.max_colwidth', None):
    print(df["review_body"][:3], '\n')

# print average length of reviews by char length AFTER preprocessing
avg_len = df["review_body"].str.len().mean()
print("Avg len after preprocessing:", round(avg_len))
```

```
[nltk_data] Downloading package wordnet to /Users/paul/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /Users/paul/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
```

1862138

wish have read review purchase do not ruin printer do not work need get photos
print be extremely disappoint

2468360 day week be automatically populate enter date be set be no way chang
e correct day week set saturday no way get say saturday will have correct day w
eek every seven years do not see advantage magnetic key holders simple hook be
force put key magnetic key ring come product need one thing pocket would not bu
y

1229465

last weeks no longer work

Name: review_body, dtype: str

Avg len after preprocessing: 201

Bigram Feature Extraction

```
In [223... import nltk
nltk.download("punkt_tab")
from nltk.util import bigrams
from nltk.tokenize import word_tokenize
from collections import Counter
from sklearn.feature_extraction import DictVectorizer

def extract_bigrams(tokens):
    bg = bigrams(tokens)
    return Counter(["_".join(pair) for pair in bg])

df["tokens"] = df["review_body"].apply(word_tokenize)

# Convert Series of Counters → list of dicts
bigram_dicts = list(df["tokens"].apply(extract_bigrams))

vec = DictVectorizer(sparse=True)
X_sparse = vec.fit_transform(bigram_dicts)
y = df["sentiment"]
```

[nltk_data] Downloading package punkt_tab to /Users/paul/nltk_data...

[nltk_data] Package punkt_tab is already up-to-date!

```
In [224... # convert to dataframe
# split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X_sparse,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

"""
total dataset size = 200,000
```

```
80% training = 160,000
20% test = 40,000
"""

print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(160000, 1640555)
(160000,)
(40000, 1640555)
(40000,)
```

Perceptron

In [225...

```
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_

perceptron = Perceptron(
    max_iter=1000,
    tol=1e-3,
    random_state=42
)

perceptron.fit(X_train, y_train)
```

Out[225...

▼ Perceptron ⓘ ?	
Parameters	
penalty	None
alpha	0.0001
l1_ratio	0.15
fit_intercept	True
max_iter	1000
tol	0.001
shuffle	True
verbose	0
eta0	1.0
n_jobs	None
random_state	42
early_stopping	False
validation_fraction	0.1
n_iter_no_change	5
class_weight	None
warm_start	False

In [226...

```
def predict_and_print_metrics(model, model_name):
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    # train metrics
    train_accuracy = accuracy_score(y_train, y_train_pred)
    train_precision = precision_score(y_train, y_train_pred)
    train_recall = recall_score(y_train, y_train_pred)
    train_f1 = f1_score(y_train, y_train_pred)

    # test metrics
    test_accuracy = accuracy_score(y_test, y_test_pred)
    test_precision = precision_score(y_test, y_test_pred)
    test_recall = recall_score(y_test, y_test_pred)
    test_f1 = f1_score(y_test, y_test_pred)

    print(model_name + " Training Accuracy: {:.4f}".format(train_accuracy))
    print(model_name + " Training Precision: {:.4f}".format(train_precision))
    print(model_name + " Training Recall: {:.4f}".format(train_recall))
```

```
print(model_name + " Training F1-score: {:.4f}".format(train_f1))

print(model_name + " Testing Accuracy: {:.4f}".format(test_accuracy))
print(model_name + " Testing Precision: {:.4f}".format(test_precision))
print(model_name + " Testing Recall: {:.4f}".format(test_recall))
print(model_name + " Testing F1-score: {:.4f}".format(test_f1))

predict_and_print_metrics(perceptron, "Perceptron")
```

Perceptron Training Accuracy: 0.9948
Perceptron Training Precision: 0.9908
Perceptron Training Recall: 0.9990
Perceptron Training F1-score: 0.9949
Perceptron Testing Accuracy: 0.8930
Perceptron Testing Precision: 0.8934
Perceptron Testing Recall: 0.8923
Perceptron Testing F1-score: 0.8929

SVM

In [227... `from sklearn.svm import LinearSVC`

```
svm = LinearSVC(
    random_state=42,
    max_iter=10000
)

svm.fit(X_train, y_train)
```

Out[227...

▼ LinearSVC ⓘ ?	
Parameters	
penalty	'l2'
loss	'squared_hinge'
dual	'auto'
tol	0.0001
C	1.0
multi_class	'ovr'
fit_intercept	True
intercept_scaling	1
class_weight	None
verbose	0
random_state	42
max_iter	10000

In [228... `predict_and_print_metrics(svm, "SVM")`

```
SVM Training Accuracy: 0.9952
SVM Training Precision: 0.9908
SVM Training Recall: 0.9997
SVM Training F1-score: 0.9952
SVM Testing Accuracy: 0.8837
SVM Testing Precision: 0.8639
SVM Testing Recall: 0.9109
SVM Testing F1-score: 0.8867
```

Logistic Regression

In [229... `from sklearn.linear_model import LogisticRegression`

```
logreg = LogisticRegression(
    random_state=42,
    max_iter=1000,
    solver='liblinear'
)

logreg.fit(X_train, y_train)
```

Out[229]...

▼ LogisticRegression ⓘ ?	
Parameters	
penalty	'deprecated'
C	1.0
l1_ratio	0.0
dual	False
tol	0.0001
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	42
solver	'liblinear'
max_iter	1000
verbose	0
warm_start	False
n_jobs	None

```
In [230] predict_and_print_metrics(logreg, "Logistic Regression")
```

```
Logistic Regression Training Accuracy: 0.9887
Logistic Regression Training Precision: 0.9804
Logistic Regression Training Recall: 0.9975
Logistic Regression Training F1-score: 0.9888
Logistic Regression Testing Accuracy: 0.8985
Logistic Regression Testing Precision: 0.8767
Logistic Regression Testing Recall: 0.9276
Logistic Regression Testing F1-score: 0.9014
```

Naive Bayes

```
In [231] from sklearn.naive_bayes import MultinomialNB

nb = MultinomialNB()

nb.fit(X_train, y_train)
```


Out[231]...

▼ MultinomialNB ⓘ ?

Parameters

alpha	1.0
force_alpha	True
fit_prior	True
class_prior	None

In [232]...

```
predict_and_print_metrics(nb, "Naive Bayes")
```

Naive Bayes Training Accuracy: 0.9457
Naive Bayes Training Precision: 0.9794
Naive Bayes Training Recall: 0.9106
Naive Bayes Training F1-score: 0.9438
Naive Bayes Testing Accuracy: 0.8688
Naive Bayes Testing Precision: 0.8870
Naive Bayes Testing Recall: 0.8452
Naive Bayes Testing F1-score: 0.8656